

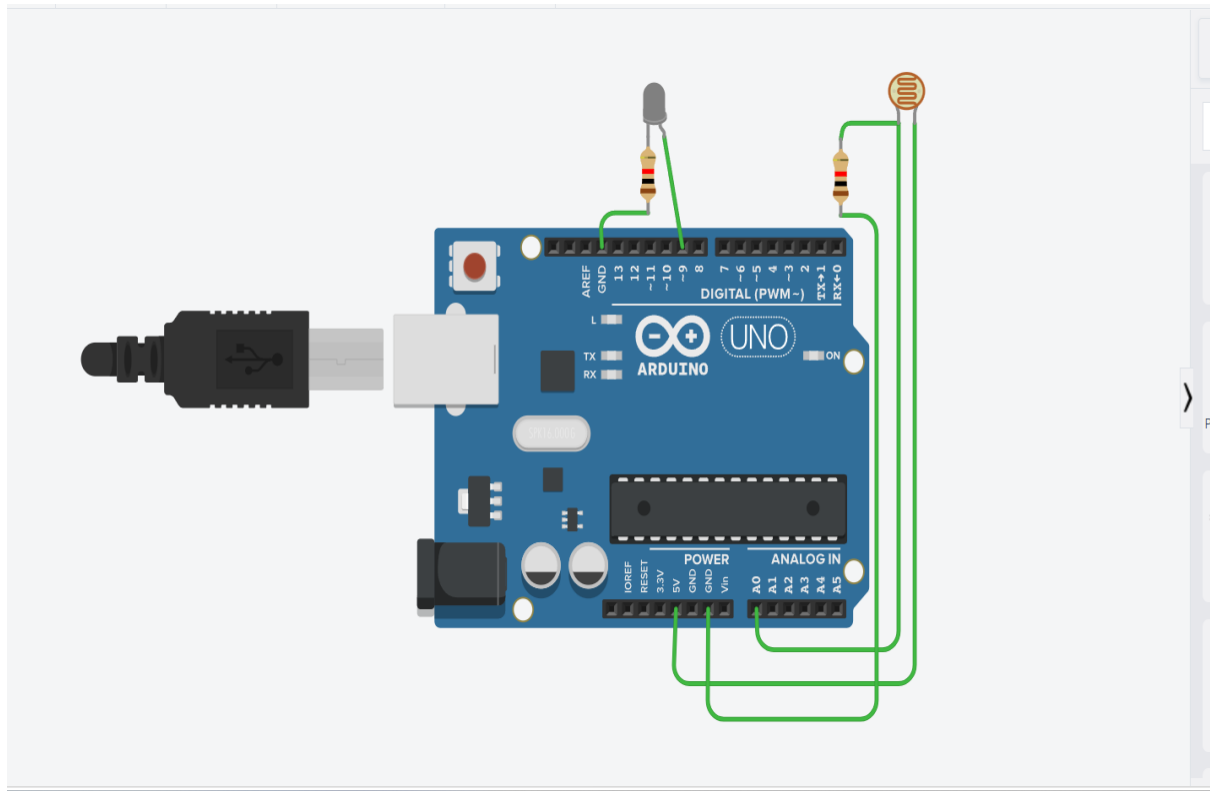
SENSOR BASED SIMULATION



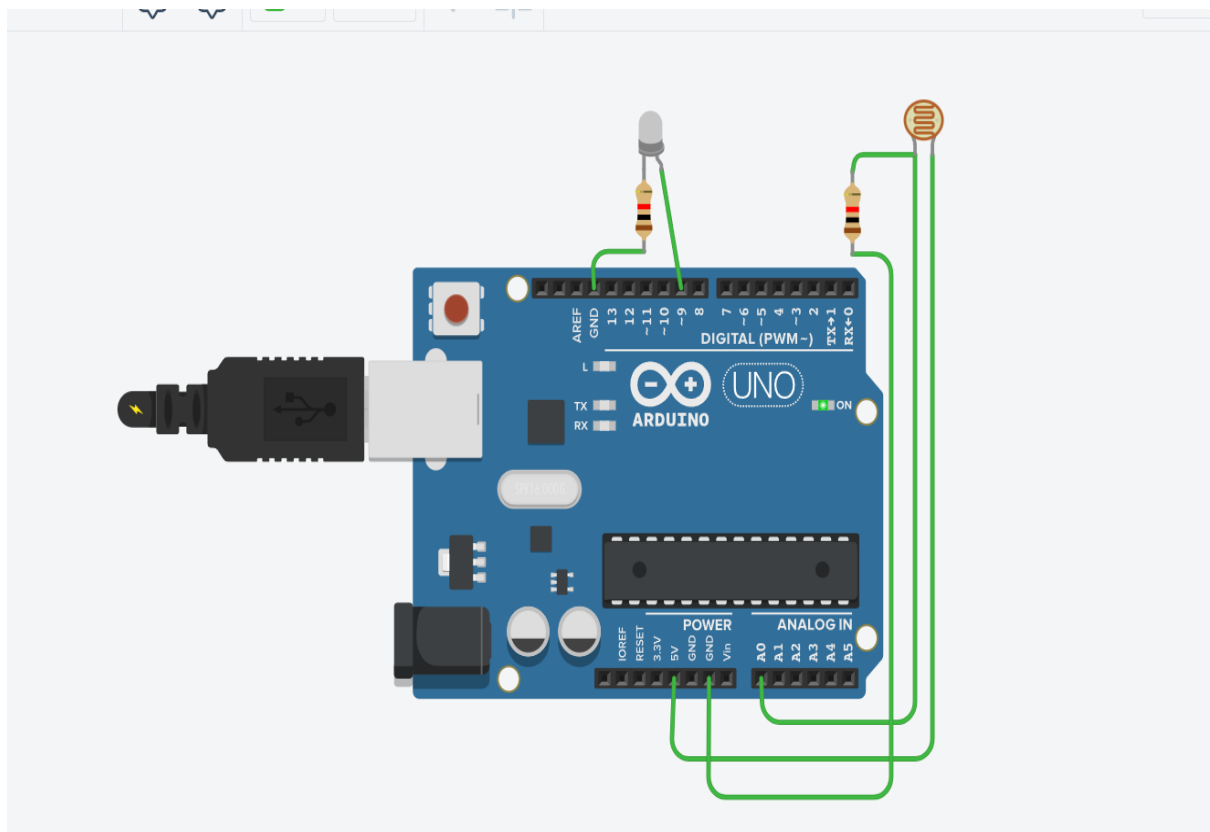
The image shows a screenshot of an IDE interface for writing and simulating Arduino code. The top toolbar includes icons for undo, redo, comments, serial monitor, and other development tools. A 'Code' button is visible on the right. Below the toolbar is a text editor with a 'Text' dropdown menu and icons for saving, opening, and font settings. The code is written in C++ and is as follows:

```
1 int sensorValue = 0;
2 void setup()
3 {
4   pinMode(A0, INPUT);
5   pinMode(9, OUTPUT);
6   Serial.begin(9600);
7 }
8 void loop()
9 {
10  sensorValue = analogRead(A0); // for reading the value from sensor
11  Serial.println(sensorValue); // to print the value that store in the sensorValue variable
12  analogWrite(9, map(sensorValue, 0, 1023, 255, 0)); // glow LED according to the LDR sensor
13  delay(100); // wait for 100 milliseconds
14 }
```

At the bottom of the IDE, there is a 'Serial Monitor' tab with a small icon of a monitor.



AFTER SIMULATION...



Code Explanation – LDR Sensor and LED using Arduino

Introduction

This project uses an LDR (Light Dependent Resistor) to detect light intensity and control an LED. The Arduino continuously reads the sensor value and turns the LED ON or OFF based on the amount of light detected.

Components Used

Arduino Uno

LDR (Light Dependent Resistor)

LED

Resistor

Connecting Wires

Program Explanation

1. Variable Declaration

The LDR sensor and LED pins are declared at the beginning of the program.

C++

```
int ldr = A0;
```

```
int led = 13;
```

A0 is used to read the LDR sensor value.

13 is used to control the LED.

2. Setup Function

The setup function runs only once when the Arduino starts.

C++

```
void setup() {  
    pinMode(led, OUTPUT);  
}
```

The LED pin is configured as an output pin.

3. Reading Sensor Value

The Arduino reads the light intensity from the LDR.

C++

```
int value = analogRead(ldr);
```

analogRead() reads the sensor value.

The value changes according to the amount of light falling on the LDR.

4. Decision Making

The sensor value is compared with a threshold value.

C++

```
if (value < 500) {  
    digitalWrite(led, HIGH);  
}  
else {  
    digitalWrite(led, LOW);  
}
```

If the light intensity is low, the LED turns ON.

If the light intensity is high, the LED turns OFF.

5. Delay Function

A small delay is added for stable operation.

C++

```
delay(100);
```

The Arduino waits for 100 milliseconds before reading the sensor again.

Working

The LDR continuously senses the surrounding light. When the environment becomes dark, the sensor value decreases and the LED turns ON. When sufficient light is present, the LED turns OFF automatically.

Result

The sensor-based simulation was successfully implemented using Arduino and LDR. The LED responded correctly to changes in light intensity, demonstrating automatic light control.

Conclusion

The project shows how an LDR sensor can be used with Arduino to create an automatic lighting system. This concept is widely used in smart street lights and home automation systems.